ReadFlow Phase 2 — Feature Prompt

Copy-ready prompt for the coding agent · Oct 25, 2026

1. Scope, and how Phase 2 fits the existing app

This prompt adds eight features to the working ReadFlow app: more fonts and reading styles with colored drop caps, opening text animations, a Figures gallery, page-flip reading, an animated dashboard with daily challenges, and swipeable flashcards with a saved deck. It extends the Folio design system and the Phase 1 rules. It does not replace them.

First step: inspect the repository, `package.json`, `DEPENDENCIES.md`, `DESIGN_NOTES.md`, `src/styles/tokens.css`, `src/motion/`, and the Dexie schema in `src/data/db.ts`. Reuse existing tokens, `Sheet`, `Cover`, `Ribbon`, `Icon`, motion variants and settings plumbing. Add nothing that already exists.

Where Phase 2 changes a Phase 1 rule

Three of your requests collide with Phase 1 rules. Resolve them this way, and record the decisions in `DESIGN_NOTES.md`.

Request	Phase 1 rule	Decision
Text animations on opening	Reading text never animates.	Animate only the **document title, chapter openers and the first paragraph's drop cap**, once, when a document or chapter opens. Body text never animates while the person reads or scrolls. A "Skip" tap and the Motion setting turn it off.
Page-flip instead of scrolling	No 3D, no heavy effects.	Offer a paged mode with three styles: Slide (default), Fade, and Flip (a light CSS 3D turn of one page, no shadows, no physics). It is a reading style, chosen in settings, never forced.

Request	Phase 1 rule	Decision
Flashcards from a page	No AI features in v1; no fabricated content.	Cards are made **without AI**: from the person's own highlights and notes, from a selected passage, or from rule-based suggestions (definitions, key sentences) that the person reviews before saving. Nothing is saved without the person's approval, and the UI never claims the suggestions are intelligent.

Hard rules that still apply

- No fake data. The dashboard shows only real reading activity from this device. Empty dashboards show an inviting empty state, never sample numbers.
- Every control works. If a feature is not built, its control is not shown.
- Local only: no backend, no analytics, no network calls with document content.
- All animation uses `transform`, `opacity` or `clip-path`, shared tokens, and honors reduced motion and lite mode.
- Out of scope: OCR, AI generation of any kind, cloud sync, accounts, notifications that need a server.

2. Fonts, reading styles and drop caps

The reader gets a **Reading style**: a named bundle of font, drop-cap treatment, heading treatment and spacing. Users pick a style in one tap, then fine-tune any single setting. The existing typography controls stay.

Reading fonts

Add these as self-hosted variable fonts through Fontsource. **Verify each package name and its OFL (or equivalent open) license before installing, record both in `DEPENDENCIES.md`, and subset to Latin plus Latin Extended (and Devanagari if the library holds Hindi documents).** Load a font only when the person selects it, with `font-display: swap`, so the first load stays small.

Font	Character	Use
Literata (existing)	Neutral bookish default	Default reading

Font	Character	Use
Source Serif 4	Crisp, slightly modern	Study and technical text
Newsreader	Editorial, elegant at large sizes	Essays, long form
Lora	Calligraphic, warm	Fiction, casual
EB Garamond	Classical, refined	Classic and literary
Crimson Pro	Compact, bookish	Small screens
Playfair Display	High contrast, dramatic	Headings and openers only, not body
Atkinson Hyperlegible Next	Maximum letter clarity	Accessibility
Lexend	Wide, even spacing	Easier reading for many readers
Instrument Sans (existing)	Clean sans	Sans option for body

Show each font in the picker as a live specimen line in its own face ("The quiet hour before dawn"), with its name beneath in the UI font. Do not render all fonts at once: render a specimen only when the picker is open, and load that font file on first view.

Reading styles (presets)

Style	Font	Drop cap	Headings	Feel
Classic	Literata	Terracotta, 3 lines, Fraunces	Fraunces, normal case	Folio default
Manuscript	EB Garamond	Ink with a thin terracotta underline, 3 lines	Playfair Display, small caps	Old book
Editorial	Newsreader	Bold theme-accent block, 2 lines	Newsreader italic	Magazine
Study	Source Serif 4	None; first word bold	Instrument Sans, medium weight	Notes and exam prep

Style	Font	Drop cap	Headings	Feel
Story	Lora	Large accent initial, 4 lines, light weight	Lora italic	Fiction
Clear	Atkinson Hyperlegible Next	None	Atkinson, bold	Accessibility, no ornament
Plain	Current font	None	Same as body, bold	Quiet and distraction-free

A style sets values; the person may change any of them afterward, and the style then shows "Custom". Persist the style and every override.

Drop caps and starting letters (the bold colored first letter)

Implement with CSS `::first-letter` on the **first paragraph after each heading and the first paragraph of each document**, not on every paragraph. A drop cap on every paragraph looks noisy; offer it as an optional "Every paragraph" choice with a small bold colored first letter (no drop) instead.

- **Modes:** Off, Drop cap (2 to 4 lines), Raised initial (large letter on the first line), Bold first word, Small caps first line.
- **Color:** Accent (default), Ink, or one of the four highlight colors, or a user-picked swatch from six curated colors. All colors are checked for 3:1 contrast in the current theme; a swatch that fails is shown disabled with the reason.
- **Weight:** Regular, Semibold, Bold, Black, where the font supports it.
- **Implementation:** `p.starts-section::first-letter` with `initial-letter: 3` where supported and a float fallback (`float: left; line-height: .8; padding-right: .08em`). Sizing and color through CSS variables (`--dropcap-lines` , `--dropcap-color` , `--dropcap-weight` , `--dropcap-font`). Handle leading quotation marks and punctuation (style the quote with the letter), and skip the drop cap for paragraphs shorter than 80 characters.
- **No re-extraction:** changing these settings updates CSS variables only and must not re-mount pages or move the first visible block.
- **Decorative style options:** line-height tweak for headings, optional ornamental divider (a small ribbon glyph, not a clip-art flourish) between sections, and a subtle rule above each folio. Keep all ornament in theme colors.

Settings UI

Add a Style tab at the top of the reading settings sheet: horizontally scrolling style cards, each a miniature live preview rendered with that style's real font and drop cap on a sample sentence. Selecting a style applies it instantly. Below, a "Customize" section with Font, Drop cap mode, Drop cap color, Weight and Heading font. Keep the sheet half-height so the text behind stays visible.

3. Opening text animations

Purpose: give opening a document the feel of opening a book, in under 1.2 seconds, without delaying reading. The animation covers only the title block and the first paragraph's initial; the rest of the text is already in place and readable underneath.

The sequence (plays once per document open, never on scroll or mode switch)

Time	What happens	How
0 to 160ms	Cover settles into the title area (existing shared-element transition).	Existing `layoutId`.
120 to 640ms	Document title reveals by line: each line rises 14px and fades in, with a 60ms stagger of at most 4 lines.	Motion variants on pre-split lines.
300 to 760ms	A hairline rule draws from left to right under the title, then the author or file name and page count fade in.	CSS `scaleX` on the rule; opacity on metadata.
500 to 1100ms	The first paragraph's drop cap scales from 0.6 to 1 and fades in, with the ribbon dropping into the top margin.	Motion `spring.tap`; transform and opacity only.

The first paragraph's body text is visible from the first frame. Only the initial letter animates. If the person scrolls or taps during the sequence, snap everything to its final state immediately.

Text reveal styles (user choice in settings: Opening animation)

- **Rise** (default): the line-by-line rise above.
- **Typewriter title:** title characters appear one by one over at most 600ms (animate `opacity` per character span, not width), with a thin caret that fades out. Titles longer than 60 characters fall back to Rise.

- **Ink fade:** the title fades from 0 to 1 with a 6px upward drift and slight letter-spacing settle (0.04em to 0em). Letter-spacing change happens only on the title, which is outside the reading flow.
- **Off.**

Also allowed and tied to the same setting: a **chapter opener** that plays the Rise style when a detected chapter heading enters the viewport for the first time in a session (heading only, 360ms, never repeated on scroll back, never for headings reached by jumping).

Implementation rules

- Split text by lines and characters in a small `SplitText` component that wraps the original text with `aria-label` and marks the animated spans `aria-hidden`, so screen readers read the title once and normally. Do not install a text-splitting package; the component is small.
- Fonts must be loaded before the sequence starts (`document.fonts.ready` with a 300ms timeout), to avoid reflow mid-animation.
- Play only for the first open of a document in a session. Opening the same document again within 5 minutes skips the sequence.
- Skip entirely when reduced motion is on, lite mode is on, the setting is Off, or the document opens at a saved position past page 1. In those cases show the final state at once.
- No layout animation on text containers; reserve the title block's final height before animating so nothing shifts.

4. Figures: finding and showing images and diagrams

Comfort mode drops pictures because it shows extracted text. Figures fixes that: the app finds images and diagrams in the PDF and shows them in a gallery and, optionally, inline where they belong. It never invents or redraws content; every figure is a crop of the original page.

Detection (new module `features/figures/`)

Run after text extraction, page by page, at low priority (yield between pages, same queue discipline as extraction), with a small "Finding figures 42 of 310" chip. Prioritize pages near the reading position and the page the person opens the gallery on.

1. **Embedded raster images.** Walk `page.getOperatorList()`; track the graphics state (`save`, `restore`, `transform`) to know each image operator's transform matrix. For `paintImageXObject`, `paintInlineImageXObject` and `paintImageMaskXObject`, the

unit square mapped by the current matrix is the figure's bounding box on the page.

2. **Vector diagrams and charts.** Many diagrams have no image at all. Detect clusters of path operations (`constructPath`, `stroke`, `fill`) with no body text inside the region: group nearby path bounding boxes (merge when gaps are under about 12 points), and keep clusters with area above about 2% of the page and at least about 12 path segments. Treat these as `diagram`.
3. **Crop from a render.** Render the page once at scale 2 (device pixel ratio capped at 2), crop each bounding box with 6 points of padding, and encode to WebP (JPEG fallback) with a 1200px maximum long edge. This preserves embedded fonts, masks and layering exactly as the original shows them.
4. **Filter noise:** drop boxes under 48 by 48 points or under 1% of page area; drop decorative lines and rules; drop the same image repeated on 40% or more of pages (logos, watermarks); drop a box covering over 85% of the page when the page also has substantial text (a scanned background or watermark); cap 12 figures per page.
5. **Captions.** Look for the nearest text line below (or above) the box whose normalized start matches `Fig`, `Figure`, `Chart`, `Diagram`, `Table` or `Exhibit` followed by a number. Store it as the caption; if none is found, leave it empty. Do not generate captions.
6. **Tables.** Tables are not detected as figures in this phase. If a figure box contains many aligned text lines, keep it as a `diagram` anyway; the crop shows it correctly.

Store each figure in the `figures` table: id, docId, pageIndex, bbox (normalized to the page), kind (`image` or `diagram`), caption, thumbnail Blob (about 360px), full Blob (1200px max), and width and height. Skip pages already processed. Make detection resumable and cancellable, and clean up canvases and operator lists after each page.

Where figures appear

- **Figures gallery (sheet or docked panel).** Opened from a new Figures button in the reader bottom bar (hidden until the document has at least one figure; shows a count). A two-column masonry layout on phones, grouped under page folios with hairlines. Filter chips: All, Images, Diagrams. Each tile shows the thumbnail, caption (two lines) and the folio. A document with no figures shows: "No images or diagrams found in this PDF."
- **Lightbox.** Tap a tile to open a full-screen viewer with pinch and double-tap zoom, pan, caption, and actions: Go to page (switches to the correct page in Comfort or Original mode), Save image (downloads the crop), Copy, and Add to notebook (saves a figure note with an optional comment). Swipe left and right moves between figures. Esc and a close button dismiss it.

- **Inline in Comfort mode.** At the position where a figure sat on its page (between the text blocks above and below by y position), show the thumbnail at a maximum height of 240px, with its caption and a "Tap to enlarge" label. Setting: Show figures inline (On by default, Off, or Gallery only). Reserve the height before the image loads so nothing shifts; use `loading="lazy"` and `decoding="async"`. Inline figures never animate while scrolling.
- **Figure bookmarks.** A saved figure appears in the Notebook under its page, with the thumbnail.
- **Library.** The document's info line shows "12 figures" when detection is complete.

Quality and honesty

- Show a small "Figures are detected automatically and may miss some" note at the bottom of the gallery.
- Provide **Rescan figures** in the document menu, and a **Add region** tool in Original mode: the person drags a rectangle over a page and it becomes a figure with a caption they can type. This handles anything detection misses and is the safety net.
- Detection failures never block reading; log them silently and continue.

5. Page-flip reading mode

Add a **Reading flow** setting: **Scroll** (existing, default) or **Paged**. Paged shows one screen of text at a time and moves by swipe, tap or key. It applies to Comfort mode. Original mode keeps its scrolling pages and gets the same swipe-to-next-page only when zoom is 1x.

Pagination (the hard part, specify carefully)

Phone screens differ, and the person changes font size, so pages must be computed, not stored.

- Render the reader's content in a fixed-height container using **CSS multi-column layout** (`column-width` equal to the reading measure, `column-gap` equal to the margin, `height` equal to the viewport minus chrome) and move between pages by translating the column strip with `transform: translate3d`. Each "page" is one column. This avoids measuring text in JavaScript and handles font changes natively.
- Paginate **one source page group at a time** (the current original page and its neighbors), not the whole document, so a 500-page book never lays out all at once. Keep a sliding window of about 3 original pages. Map positions as (pageIndex, blockIndex, characterOffset) so that the person's place survives a font-size change, and recompute after any typography change by scrolling the column strip to the page that contains the saved offset.

- Show "page 7 of 12 in this section" locally and the original folio globally ("p. 42 of 310"). Do not claim a global flipped page count; it is not stable.
- Figures inline are allowed in Paged mode; a figure taller than the page opens in the lightbox instead of splitting.
- Highlights, notes and bookmarks work as in Scroll mode. Selection can start on a page and end at the page edge only; extending across a flip is not supported, and the selection bar says so if the person tries.

Flip styles (setting: Page turn)

Style	Behavior	Implementation
Slide (default)	The strip follows the finger and settles on the next column.	Motion `drag="x"` with `dragConstraints`, spring release (`spring.sheet`), snap at 30% width or velocity over 400px/s.
Fade	A short crossfade between pages.	Two layers, opacity 0 to 1 over `--dur-base`.
Flip	The outgoing page rotates around the left edge to reveal the next, in a light book-like turn.	CSS 3D: a wrapper with `perspective: 1400px`, the outgoing page `rotateY` from 0 to -90deg over 320ms with `transform-origin: left center` and `backface-visibility: hidden`, then the next page appears. No curl, no gradient shading, no shadow animation. Skip the 3D (fall back to Slide) in lite mode and under reduced motion.

Controls

- **Tap zones:** right 35% next page, left 35% previous page, center toggles chrome. Zone widths adjustable (Normal, Wide center) in settings. Never trigger a flip on a tap that begins on a link, figure or highlight.
- **Swipe:** horizontal drag. A vertical drag is ignored in Paged mode and never scrolls.
- **Keyboard:** Right, Space and Page Down next; Left and Page Up previous; Home and End jump to the start and end of the document.
- **Volume buttons:** not available in browsers; do not promise them.
- **Scrubber:** the existing page scrubber works, and releases to the exact paged position.
- **Progress:** the top ribbon line updates after each flip; page dots are not used.

- Preload the next and previous page layers when idle so a flip never waits on layout. Use `will-change: transform` only during an active gesture.
- Remember the person's choice per document and as a default.

Accessibility

Each page is a labeled region ("Page 7 of 12 in this section"); next and previous are real buttons with accessible names and are the keyboard fallback for any gesture. Announce the folio through a polite live region after a flip. Respect reduced motion by switching Flip and Slide to an instant swap with a 120ms fade.

6. Animated dashboard, daily stats, challenges

A new **Today** screen (`/today`) turns real reading activity into a calm, animated daily view. Navigation becomes a bottom bar on phones and a slim left rail on desktop with three destinations: **Library**, **Today**, **Cards**. Keep it quiet: icon plus label, a ribbon-colored underline on the active item, no badges except a count of due flashcards.

What is measured (real data only)

Add a `sessions` table: id, docId, dateKey (local `YYYY-MM-DD`), startedAt, endedAt, activeSeconds, pagesRead, wordsEstimate.

- **Active time:** count a second only while the reader is visible, the tab is focused, and the person interacted (scroll, tap, key, flip) in the last 60 seconds. Pause when idle, when the tab is hidden, or when audio plays but the reader is untouched.
- **Pages read:** an original page counts once per session when it stays in view for at least 3 seconds. Skipping through with the scrubber does not count.
- **Words:** estimated as characters of extracted text on counted pages divided by 5. Always label as "about".
- Write on `visibilitychange` , `pagehide` and every 15 seconds while active. A crash loses at most 15 seconds.
- **Streak:** consecutive local days on which the daily goal was met. A missed day resets it. No "freeze" or fabricated forgiveness.
- All data stays on the device. Provide Export (JSON) and Reset stats (with confirmation) in Today's menu.

Screen composition (not a card dashboard)

Use the Folio language: large serif numerals, hairline rules, the ribbon motif. No grid of identical boxes.

1. **Today hero.** A thin circular arc (stroke, not a filled donut) around a large Fraunces numeral showing minutes read today, with "of 20 min goal" beneath. Beside it: pages today and words about. A one-line status in plain words ("12 minutes to your goal", "Goal met. Nicely read."). First visit asks the person to choose a daily goal (10, 20, 30, 45 minutes or custom); store it. Never preset a fake history.
2. **This week.** Seven vertical ribbon bars (swallowtail bottoms), height proportional to minutes, today's ribbon in the accent color and others in `--ink-3`. Day initials beneath; tapping a bar shows that day's minutes and pages in a small readout. Hand-rolled SVG.
3. **Currently reading.** Up to three in-progress documents as covers with title, "page 42 of 310", ribbon progress, a Continue button, and "about 3 h left" only after the person has at least 3 sessions (derived from their own pace, labeled estimated).
4. **Daily challenges.** See below.
5. **Last 12 weeks.** A calendar heat map of minutes per day, five tint steps of the accent, with a legend. Empty days stay as hairline outlines.
6. **Totals.** Documents opened, pages read, hours read, longest streak, flashcards reviewed. Plain text rows separated by hairlines.

Empty state (no sessions yet): the Today hero shows `0` and "Open a document and your reading will appear here." with a Continue or Import button; no placeholder charts.

Daily challenges and routine

Two parts, both fully local.

Daily challenges (generated from real context, deterministic per day). Each day show up to three, selected by a seeded pick from the eligible templates using the date as the seed, so the list is stable all day and not random on refresh. Eligibility is checked against real state, so no impossible challenges appear.

Template	Eligible when	Completes when
Read for N minutes	Always (N from the goal, or goal plus 10 as a stretch)	Active minutes reach N
Read N pages	At least one document in progress	Pages read reach N
Finish a chapter or a section	A document has detected headings	The reader crosses into the next heading
Highlight N passages	A document in progress	N highlights created today
Review N flashcards	Deck has N due cards	N reviews today

Template	Eligible when	Completes when
Revisit an old document	A document untouched for 7 days or more	It is opened for 2 minutes
Add a figure to your notebook	A document has figures	A figure note is saved

Completion is detected automatically from activity, never self-reported. A completed challenge shows a drawn checkmark and moves to the bottom.

Personal tasks and routine. A simple task list the person writes: title, optional note, optional repeat (Daily or Weekdays), optional link to a document. Repeating tasks reset each local day and keep a per-task streak. Tasks are checked by tapping; completion is the person's own. Reorder by long-press drag, delete with Undo. Offer starter suggestions as optional chips the person taps to add ("Revise yesterday's highlights", "Read before bed"); never added automatically.

Dashboard motion

- Numerals count up from 0 once when the screen first enters view (600ms, `--ease-out`, tabular figures), not on every revisit within the session.
- The goal arc draws with `stroke-dashoffset` (CSS) over 700ms; the ribbon bars rise from the baseline with a 30ms stagger, `scaleY` from the baseline with `transform-origin: bottom`.
- Heat map cells fade in by row, 12ms stagger, once.
- Meeting the goal plays one moment: the ribbon drops into the hero and the arc closes with a 300ms pulse. No confetti, no sound.
- Checking a task draws the checkmark (160ms) and strikes the title with a `scaleX` line.
- Reduced motion: values appear instantly, arcs and bars render at final size, no pulse.

7. Flashcards: swipe learning and saved deck

The person turns what they read into colorful cards and learns them by swiping. No AI is used. The app proposes cards by simple text rules, the person reviews and edits each one, and only approved cards are saved.

Making cards (module `features/cards/`)

Entry points

1. **Make cards from this page:** a button in the reader's page menu (and a Cards icon in the bottom bar overflow). Opens the Card Maker sheet for the current original page.

2. **From a selection:** add a fifth action, Card, to the selection bar. It opens the maker with the selected text as the back and a blank front.
3. **From highlights and notes:** in the Notebook, each highlight has Make card. Front defaults to the person's note if one exists, otherwise to the heading above it and the folio ("Under *Photosynthesis*, p. 42"); back is the highlighted text.
4. **Manual:** New card in a deck.

Card Maker sheet. Shows the page's suggestions as a list of editable cards, each with a checkbox (all unchecked by default, so nothing is saved unreviewed), a Front field, a Back field, a type label, and a color dot. Footer: deck picker (default: a deck named after the document, created on first save), "Add N cards". Maximum 12 suggestions per page. If the page produces none, say: "No obvious cards on this page. Select text to make one yourself."

Rule-based suggestions (no AI, state this in a small caption: "Suggested by simple rules. Check each card.")

Rule	Detects	Card
Definition	Sentences like "X is/are/means/refers to Y" and lines like "Term: explanation" where the term is 6 words or fewer	Front: "What is X?" Back: the definition sentence, unaltered
Bold term	Text items whose font name contains Bold, followed by an explanation in the same paragraph	Front: the term. Back: the paragraph
Fact	Sentences that contain a year, date, percentage or other number with a named subject	Cloze: the number is blanked in the front; the back shows the full sentence
Key sentence	The first sentence of a paragraph under a detected heading	Front: the heading as a prompt. Back: the sentence
List	Consecutive bulleted or numbered lines under a lead line	Front: the lead line. Back: the list

Back text is always the document's own wording; do not paraphrase or generate text. Fronts built by templates ("What is X?") are editable. Provide Cloze with a `{{blank}}` marker the person can move by selecting a different word.

Decks and storage

New Dexie tables: `decks` (id, name, color, docId?, createdAt), `cards` (id, deckId, type, front, back, color, docId, pageIndex, blockIndex, saved, box, dueAt, createdAt), `reviews` (id, cardId, at, result, dateKey). Cards link to their source so **Go to source** jumps to the page and passage in the reader.

Cards screens

- **Deck list (`/cards`).** Decks shown as small stacks of colored cards (two offset layers behind the top card), with name, total cards, due today, and saved counts. A **Saved** smart deck sits first, gathering every saved card across decks. A New deck button; menu per deck: Rename, Color, Export as CSV, Delete with Undo.
- **Deck view.** Segments: Study, Browse. Browse is a grid of cards that flip on tap, with edit and delete.
- **Study (`/cards/:deckId/study`).** Full-screen, one card at a time (see below).
- **Session summary.** Counts for Got it, Again and Saved (count-up numerals), a ribbon drop, and two actions: "Study missed cards" and "Done".

Swipe learning

- **The card:** a large, colored face (see palette) with the front text centered in Literata (size adjusts to fit, minimum 18px), a small folio and source at the bottom, and a type label. Tap or press Space to flip with a 3D `rotateY` over 240ms; the back shows the answer in the card's own color family.
- **Gestures** (Motion `drag`, `dragElastic: .6`, `useTransform` for tilt and overlays): **right = Got it**, **left = Again**, **up = Save for later** (toggles Saved). Dragging tilts the card (`rotate` up to 10 degrees) and fades in a label ("Got it", "Again", "Saved") plus a color wash overlay in sage, clay or saffron. Release past 120px or 500px/s commits and the card flies off; otherwise it springs back.
- **Buttons and keys (required, not optional):** Again, Got it, Save buttons under the card; Left, Right, Up arrow keys; F or Space to flip; Esc to exit. Every gesture has a button.
- **Stack:** show the next two cards behind, scaled .96 and .92 and offset 8px. The next card rises into place on commit.
- **Progress:** a ribbon bar at the top ("7 of 15"), tabular figures.
- **Undo:** a Back button restores the last card and its previous scheduling.
- **Scheduling (simple Leitner boxes, local, deterministic):** boxes 1 to 5 with intervals of 1, 2, 4, 8 and 16 days. Got it moves up one box; Again returns to box 1 and re-queues the card at the end of the current session. A session shows cards due today plus new

cards, up to a session size the person picks (10, 20, all). Label it "Review schedule", not "smart" or "AI".

- **Revisit saved cards:** the Saved deck and a "Saved" filter in Browse; a saved card keeps its schedule and also appears in a "Saved" study session on demand.

Colorful cards (tokens)

Define eight card colors in `tokens.css` as `--card-1` to `--card-8` with matching `--on-card-N`, tuned per theme and verified at 4.5:1: Terracotta #C8553D, Saffron #E3A72F (dark text), Sage #6E9B7B, Slate #5B7FA3, Plum #8A5A83, Teal #3F8F8B, Rose #D9778A (dark text), Ink #2B2723. In dark themes lower saturation about 10% and keep text contrast. A deck has a base color; cards inherit it and may override it. No gradients and no glow; color is a flat fill with a hairline inner border.

Motion for cards

Flip `rotateY` 240ms with `backface-visibility: hidden` and `perspective: 1000px` on the card container only. Commit fly-off uses `spring.sheet`. Overlays fade with the drag distance and need no timers. Summary numerals count up once. Reduced motion: flip becomes a 120ms crossfade, fly-off becomes a fade, tilt is disabled, buttons remain.

8. Data model, architecture and dependencies

Dexie migration (version 2)

Bump the schema with a proper `db.version(2).stores({...}).upgrade(...)`. Existing documents, highlights, bookmarks, progress and settings must survive untouched; test the upgrade against a database created by the Phase 1 build.

Table	Key fields	Used by
`figures`	id, docId, pageIndex, bbox, kind, caption, thumbBlob, fullBlob, w, h	Figures
`sessions`	id, docId, dateKey, startedAt, endedAt, activeSeconds, pagesRead, wordsEstimate	Dashboard
`goals`	key, value (dailyMinutes, startedOn)	Dashboard
`tasks`	id, title, note, repeat, docId?, order, createdAt, archived	Personal tasks
`taskLog`	id, taskId, dateKey, doneAt	Task streaks

Table	Key fields	Used by
`challengeLog`	dateKey, challengeId, completedAt	Daily challenges
`decks`	id, name, color, docId?, createdAt	Flashcards
`cards`	id, deckId, type, front, back, color, docId, pageIndex, blockIndex, saved, box, dueAt, createdAt	Flashcards
`reviews`	id, cardId, at, result, dateKey	Flashcards and stats

Index fields used in queries (`dateKey` , `dueAt` , `deckId` , `[docId+pageIndex]`). Add figure-note and card references to the existing notebook entries through a `kind` field rather than a new notebook table.

Settings additions (in the existing `settings` table): `readingStyle` , `readingFont` , `dropCapMode` , `dropCapColor` , `dropCapWeight` , `headingFont` , `openingAnimation` , `readingFlow` (scroll or paged), `pageTurn` , `tapZones` , `figuresInline` , `cardSessionSize` .

New and changed modules

```
src/features/
  typography/    ReadingStyles, FontPicker, DropCapControls, styles.ts
(presets)
  opening/       SplitText, OpeningSequence, useOpeningPlayed
  figures/       detectFigures, cropFigure, FiguresSheet, Lightbox, AddRegion
  paged/         PagedReader, paginate (CSS columns), flip styles, tapZones
  today/         TodayPage, GoalArc, WeekRibbons, HeatMap, Challenges, Tasks
  tracking/      sessionTracker (active-time logic), stats queries
  cards/         CardMaker, suggestRules, DeckList, StudyView, CardStack,
scheduler
  nav/           BottomBar, SideRail (Library, Today, Cards)
src/motion/      add: countUp, drawPath, cardFlip, swipe thresholds
```

The tracker is a single module with a small public API (`start(docId)` , `touch()` , `stop()`), mounted once in the reader, and tested without UI.

Dependencies

Inspect first; add only what is missing. **Verify names and licenses and record versions in `DEPENDENCIES.md` .**

- **Add:** Fontsource variable packages for the new fonts you ship (Source Serif 4, Newsreader, Lora, EB Garamond, Crimson Pro, Playfair Display, Atkinson Hyperlegible Next, Lexend), loaded lazily.
- **No new animation, chart, text-splitting, flip-book, drag-and-drop or card-swipe library.** Motion already covers drag, springs, layout and `useTransform`; the charts are small hand-built SVG; page flip uses CSS; text splitting is a 40-line component. Libraries such as `react-pageflip`, `react-tinder-card`, Chart.js, Recharts and GSAP SplitText are explicitly excluded.
- **Optional, only if reorder by drag proves hard with Motion's `Reorder`:** nothing new; use `Reorder` from `motion/react` first.
- **Do not add** an AI SDK, a spaced-repetition package with network sync, or any analytics package.

If you believe a new package is justified, write the reason, the measured size cost and the alternative in `DEPENDENCIES.md` before installing.

9. Motion, performance and accessibility rules

Elegance rules for every new screen

- Same Folio language: ribbon, folio numerals, hairline rules, serif voice. New features may add color only through the card palette and the drop-cap color, both token-based.
- One focal motion per screen at a time. The dashboard animates on first entry and then goes still.
- Add each new motion to `src/motion/` as a token or variant with a stated purpose; no inline durations.
- Write a one-line rationale for each new screen and component in `DESIGN_NOTES.md`.

Performance budgets (measure and record in `PERFORMANCE.md`)

Area	Target
Fonts	A font file loads only when chosen or previewed; first-load font weight does not grow versus Phase 1.
Figure detection	No main-thread task over 50ms; a 300-page document finishes in the background without a scroll frame drop below 55fps at 4x CPU throttle.

Area	Target
Paged mode	Flip responds within 100ms; pagination of a 3-page window under 60ms on a mid-range phone profile; no layout shift after a flip.
Dashboard	Opens in under 300ms from cached data; animations use only `transform`, `opacity`, `stroke-dashoffset`.
Swipe cards	Drag stays at 60fps; at most 3 cards mounted at once.
Storage	Figures cap at about 1200px WebP; show used storage in settings and offer Delete figures per document.
Bundle	Today, Cards, Figures and Paged load as lazy chunks; the library and reader entry chunk grows by less than 15 KB gzip.

Accessibility

- Every gesture (swipe cards, page flip, lightbox zoom, task reorder) has a visible button and a keyboard path.
- Drop-cap letters and split title text must not break reading by screen readers: animated spans are `aria-hidden`, the full text is available once. `initial-letter` and floats must not change the DOM text.
- Chart data on Today has a text equivalent: a visually hidden table of the week's minutes, and an accessible name on the goal arc ("14 of 20 minutes today").
- Challenge and task state uses icon plus text, never color alone. Card colors and drop-cap colors pass 3:1 against the page and 4.5:1 for text on cards.
- Live regions announce flips ("Page 7 of 12"), swipe results ("Got it, 8 of 15") and goal completion.
- Reduced motion and lite mode behavior is defined per feature in sections 3 to 7 and must be tested.
- Touch targets stay 44 by 44px; the Cards bottom bar buttons are at least 56px tall.

Failure handling

Guard every new Dexie call with try/catch and show a plain message for quota errors. Tracking, figure detection and card suggestion failures never interrupt reading. Clean up observers, timers, canvases and drag listeners on unmount.

10. Phases, testing and acceptance criteria

Build in this order. After each phase run the build, check the exit condition, commit, and add two lines to `PROGRESS.md`. Do not start a phase with a broken build.

Phase	Build	Exit check
0. Inspect	Read the repo and docs listed in section 1. Write the Phase 2 plan in `DEPENDENCIES.md`.	Plan lists only needed additions.
1. Fonts and styles	Lazy fonts, font picker with specimens, reading styles, drop-cap system and controls, per-style previews.	Style switch is instant; first visible block does not move; contrast checks pass.
2. Opening animations	`SplitText`, opening sequence, Opening animation setting, chapter openers, skip rules.	Plays once; body text never animates; screen reader reads the title once.
3. Figures	Detection, cropping, `figures` table, gallery, lightbox, inline figures, Add region, rescan.	Fixtures with raster and vector figures are found; noise is filtered.
4. Paged mode	CSS-column pagination, three page turns, tap zones, position mapping, scrubber.	Font size change keeps the reading place; highlights work in Paged mode.
5. Tracking and Today	Session tracker, goal, hero arc, week ribbons, heat map, current reading, totals, motion.	Numbers match a manual stopwatch test within 5 seconds per 5 minutes.
6. Challenges and tasks	Eligibility rules, seeded daily pick, auto completion, tasks with repeat and streaks.	Challenges update from real activity; list is stable on refresh.
7. Flashcards	Card Maker with rules, decks, Browse, Study with swipe and buttons, scheduler, Saved deck, summary.	A page becomes reviewed cards; saved cards reappear; due counts correct.
8. Polish	Navigation bar and rail, empty states, motion pass, reduced-motion and lite mode, README update.	Section 9 budgets met or explained.

Testing

Fixtures (generated by script under `tests/fixtures/`): a PDF with embedded raster images and captions; one with vector charts; one with definitions, bold terms, dates and lists for card rules; a long PDF with chapter headings; and a database snapshot from Phase 1 for migration.

Unit tests: drop-cap eligibility and paragraph selection; font and style preset resolution; figure bounding box math and noise filters; caption matching; pagination position mapping across a font-size change; active-time counter (idle, hidden tab, interaction); streak and dateKey logic across midnight and time-zone change; challenge eligibility and seeded pick stability; Leitner scheduling; card suggestion rules (including that back text equals source text); Dexie v1 to v2 upgrade.

End-to-end (mobile profile plus desktop): switch every reading style and font; opening animation plays once and can be skipped; figures gallery and lightbox with Go to page; Paged mode with swipe, tap zones, keyboard, and a font change mid-chapter; read for two minutes and see Today update; complete a challenge by activity; add, repeat and check a task; make cards from a page, study with swipe and with buttons, save a card, finish a session; reload and confirm all data persists; repeat key flows with reduced motion and with keyboard only.

Accessibility and performance: axe on Today, Cards, Study, Figures, Card Maker and reading settings in every theme; Lighthouse mobile; Performance panel at 4x throttle for figure detection, Paged flips and swipe cards. Record results in `A11Y.md` and `PERFORMANCE.md`.

Acceptance criteria

Reading

- [ ] At least eight reading fonts and seven reading styles work, load lazily, and persist.
- [ ] Colored, bold drop caps and the other starting-letter modes work with color, weight and size controls, never alter the text content, and never shift the reading position.
- [ ] The opening sequence plays once per open, never animates body text, and is skippable; it is off under reduced motion and lite mode.
- [ ] Paged mode paginates correctly, survives typography changes, and offers Slide, Fade and Flip with tap, swipe and keyboard control.

Figures

- [ ] Images and vector diagrams are found, cropped from the page, filtered for noise, shown in the gallery and lightbox, and optionally inline; Add region catches misses.

Dashboard

- [ ] Today shows only real activity; empty states appear for new users; goal, week, heat map and totals match recorded sessions.
- [ ] Daily challenges are eligible, stable per day and complete automatically; tasks repeat and keep streaks.
- [ ] Dashboard animations run once on entry, use only allowed properties, and respect reduced motion.

Flashcards

- [ ] Cards can be made from a page, a selection, a highlight and by hand; suggestions are reviewed and never auto-saved; back text matches the source.
- [ ] Swipe study works by gesture, buttons and keys; scheduling, undo, Saved deck and session summary work; decks persist and link back to the source page.

Quality

- [ ] Phase 1 data migrates intact; all Phase 1 features still pass their tests.
- [ ] `npm run build` passes with no type errors; no console errors on the main flows.
- [ ] Axe shows zero serious or critical issues in all themes; performance numbers are recorded and meet section 9 or are explained.
- [ ] No fake data, no placeholder controls, no new excluded dependencies, and `README.md`, `DESIGN_NOTES.md` and `DEPENDENCIES.md` are updated.